

Ayúdame a demostrar el siguiente ejercicio de teoría de la complejidad. Define todo, sé detallado y da ejemplos.

Muestre que existe $L \in CFL$ tal que L es NL -Completo

Ha pensado durante 16s

Sí. Este ejercicio es más sutil de lo que parece porque **no basta con encontrar un lenguaje libre de contexto**: tenemos que construir uno que además esté en NL y sea NL -hard bajo reducciones en espacio logarítmico.

La construcción clásica usa una versión por niveles de **PATH**. Recordemos que la alcanzabilidad dirigida es NL -completa.  Complejidad Zoo +1 La existencia de un lenguaje incluso **lineal** —y por tanto context-free— que es NL -completo es un resultado conocido. 

1. ¿Qué debemos demostrar?

El enunciado pide:

$$\exists L \in CFL \text{ tal que } L \text{ es } NL\text{-completo.}$$

Recordemos las definiciones.

CFL

CFL , *Context-Free Languages*, es la clase de lenguajes que pueden ser reconocidos por un **autómata de pila (PDA)** o, equivalentemente, generados por una gramática libre de contexto.

NL

$$NL = NSPACE(O(\log n)).$$

Es decir, un lenguaje A pertenece a NL si existe una máquina de Turing **no determinista** que decide A utilizando

$$O(\log n)$$

espacio de trabajo.  Complejidad Zoo

***NL*-completo**

Un lenguaje L es *NL*-completo si se cumplen dos condiciones:

$$L \in NL$$

y

$$\forall A \in NL, \quad A \leq_L L$$

donde $\leq_L$ significa reducción many-one computable usando $O(\log n)$ espacio.

Por tanto nuestra demostración tendrá la estructura:

$$L \in CFL, \quad L \in NL, \quad L \text{ es } NL\text{-hard.}$$

2. Partimos del problema PATH

Usaremos el problema

$$PATH = \{\langle G, s, t \rangle : \text{existe un camino dirigido de } s \text{ a } t\}.$$

Por ejemplo:

$$s \rightarrow a \rightarrow b \rightarrow t$$

implica

$$\langle G, s, t \rangle \in PATH.$$

PATH es uno de los problemas canónicos *NL*-completos: una máquina no determinista puede mantener únicamente el vértice actual y adivinar sucesivamente el siguiente vértice del camino.

⊕ Wikipedia

Así que para demostrar *NL*-hardness bastará demostrar

$$PATH \leq_L L.$$

Pero no reduciremos directamente. Introduciremos primero un problema intermedio.

3. Grafos por niveles

Consideremos un grafo dirigido cuyos vértices están separados en niveles

$$A_1, A_2, \dots, A_k$$

y donde las aristas solamente pueden ir del nivel i al nivel $i + 1$:

$$A_i \longrightarrow A_{i+1}.$$

Por ejemplo:

$$\begin{array}{cccc} A_1 & A_2 & A_3 & A_4 \\ s & a & c & t \\ & b & d & \end{array}$$

con posibles aristas

$$s \rightarrow a, \quad s \rightarrow b, \quad a \rightarrow c, \quad b \rightarrow d, \quad c \rightarrow t.$$

Definimos

$$LEVELED-PATH = \{ \langle G, s, t \rangle : G \text{ es por niveles y existe un camino } s \rightsquigarrow t \}.$$

La solución clásica usa precisamente esta variante.  studylib.net

4. LEVELED-PATH es NL -completo

Primero veamos que

$$PATH \leq_L LEVELED-PATH.$$

Supongamos que el grafo original G tiene m vértices.

Construimos m copias de cada vértice:

$$[v, 1], [v, 2], \dots, [v, m].$$

El segundo componente indica el nivel.

Así,

$$A_i = \{[v, i] : v \in V\}.$$

Si existe una arista

$$u \rightarrow v$$

en G , introducimos en el nuevo grafo las aristas

$$[u, i] \rightarrow [v, i + 1], \quad i = 1, \dots, m - 1.$$

Además añadimos

$$[t, i] \rightarrow [t, i + 1].$$

Estas últimas aristas sirven para "esperar" en t .

La construcción usada para demostrar la completitud de LEVELED-PATH es justamente ésta.

⊕ studylib.net

¿Por qué funciona?

Si

$$s = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_r = t$$

es un camino en G , entonces tenemos

$$[s, 1] \rightarrow [v_2, 2] \rightarrow \dots \rightarrow [t, r].$$

Después utilizamos las aristas

$$[t, r] \rightarrow [t, r + 1] \rightarrow \dots \rightarrow [t, m].$$

Por tanto

$$s \rightsquigarrow t \text{ en } G$$

si y solo si

$$[s, 1] \rightsquigarrow [t, m]$$

en el grafo por niveles.

La transformación puede generarse usando $O(\log m)$ bits para almacenar los índices de los vértices y niveles.

Por consiguiente,

$$PATH \leq_L LEVELED-PATH.$$

Como PATH es NL -completo,

$$\boxed{LEVELED-PATH \text{ es } NL\text{-hard.}}$$

Y evidentemente está en NL , porque podemos adivinar un vértice del siguiente nivel.

Luego

$$\boxed{LEVELED-PATH \text{ es } NL\text{-completo.}}$$

5. Una pequeña modificación: POINTED-LPATH

Nos conviene exigir que el primer y último nivel tengan un único vértice.

Definimos

$$POINTED-LPATH$$

como los grafos por niveles tales que

$$A_1 = \{s\}$$

y

$$A_k = \{t\}.$$

Además preguntamos si existe

$$s \rightsquigarrow t.$$

Esto no cambia la dificultad.

Dado un LEVELED-PATH, simplemente podemos agregar:

$$s' \rightarrow s$$

y

$$t \rightarrow t'$$

creando un nuevo primer y último nivel:

$$\{s'\}, A_1, A_2, \dots, A_k, \{t'\}.$$

Por tanto

$$LEVELED-PATH \leq_L POINTED-LPATH.$$

Así,

$$\boxed{POINTED-LPATH \text{ es } NL\text{-hard.}}$$

Esta transformación también forma parte de la construcción estándar.  studylib.net

6. Ahora construimos nuestro lenguaje L

Aquí está la idea central.

Supongamos que cada vértice se representa mediante una cadena binaria.

Por ejemplo,

$$a = 00, \quad b = 01, \quad c = 10, \quad d = 11.$$

Representaremos una arista

$$u \rightarrow v$$

mediante el par

$$\boxed{(u, v^R)}$$

donde

$$v^R = Rev(v)$$

es la cadena v escrita al revés.

Por ejemplo, si

$$u = 001, \quad v = 110,$$

representamos la arista como

$$(001, 011).$$

¿Por qué invertimos v ?

Porque queremos usar **una pila**.

7. El lenguaje

Una palabra de L estará dividida en bloques:

$$B_1; B_2; \dots; B_r.$$

Cada bloque contiene una lista de pares:

$$B_i = (x_{i1}, y_{i1}), (x_{i2}, y_{i2}), \dots, (x_{il_i}, y_{il_i}).$$

Definimos

$$L = \left\{ B_1; \dots; B_r : \begin{array}{l} \exists i_1, \dots, i_r \text{ tales que} \\ \text{Rev}(y_{j, i_j}) = x_{j+1, i_{j+1}} \\ \text{para todo } j < r \end{array} \right\}.$$

En palabras:

Podemos escoger **un par de cada bloque** de manera que el destino de la arista escogida en un bloque coincida con el origen de la arista escogida en el siguiente.

Esta es la construcción clásica del lenguaje C utilizado para obtener un CFL NL -completo.

8. Ejemplo para entender L

Consideremos el grafo

$$s \rightarrow a \rightarrow c \rightarrow t.$$

Supongamos las codificaciones

$$s = 00, \quad a = 01, \quad c = 10, \quad t = 11.$$

También podría haber otras aristas:

$$s \rightarrow b, \quad b \rightarrow d.$$

El primer bloque contiene las aristas que salen del primer nivel:

$$B_1 = (00, Rev(01)), (00, Rev(11)).$$

El segundo bloque podría ser

$$B_2 = (01, Rev(10)), (11, Rev(00)).$$

Y el tercero

$$B_3 = (10, Rev(11)), (00, Rev(01)).$$

Podemos escoger

$$(00, Rev(01)),$$

luego

$$(01, Rev(10)),$$

y finalmente

$$(10, Rev(11)).$$

Observe las coincidencias:

$$Rev(Rev(01)) = 01,$$

que es el origen de la segunda arista, y

$$Rev(Rev(10)) = 10,$$

que es el origen de la tercera.

Por tanto hemos codificado

$$00 \rightarrow 01 \rightarrow 10 \rightarrow 11,$$

es decir,

$$s \rightarrow a \rightarrow c \rightarrow t.$$

9. ¿Por qué $L \in CFL$?

Ésta es una parte fundamental.

Construiremos conceptualmente un **PDA no determinista**.

Supongamos que está leyendo

$$B_1; B_2; \dots; B_r.$$

Dentro de B_1 , el PDA **adivina no determinísticamente** cuál par quiere seleccionar:

$$(x, y).$$

Ignora x y, mientras lee y , introduce sus símbolos en la pila.

Supongamos

$$y = a_1 a_2 \dots a_m.$$

La pila queda, desde arriba,

$$a_m, \dots, a_2, a_1.$$

Ahora pasa al siguiente bloque.

El PDA nuevamente adivina qué par

$$(x', y')$$

quiere escoger.

Mientras lee x' , compara cada símbolo contra el tope de la pila.

Aceptará esa comparación exactamente cuando

$$x' = Rev(y).$$

Después, cuando lee y' , lo introduce en la pila para compararlo con el siguiente bloque.

Así procede:

$\text{elegir} \rightarrow \text{apilar} \rightarrow \text{comparar} \rightarrow \text{apilar} \rightarrow \text{comparar} \rightarrow \dots$

Por ejemplo, si aparece

$(\dots, 001)$

el PDA introduce

$0, 0, 1.$

Por tanto la pila permite leer después

$100.$

Es decir,

$$100 = \text{Rev}(001).$$

Así el PDA verifica exactamente la condición

$$\text{Rev}(y_j) = x_{j+1}.$$

Como existe un PDA que reconoce L ,

$$L \in CFL.$$

10. ¿Por qué $L \in NL$?

Cuidado con este punto:

$$L \in CFL$$

no implica automáticamente

$$L \in NL.$$

De hecho, CFL se sitúa naturalmente dentro de $LOGCFL$, y se sabe que $NL \subseteq LOGCFL$; no se identifica en general CFL con NL . $\oplus$ Complejidad Zoo +1

Por eso debemos demostrar específicamente que nuestro L pertenece a NL .

La máquina NL hace lo siguiente.

Para cada bloque B_j :

1. adivina no determinísticamente cuál par escoger;
2. almacena únicamente la **posición** de ese par;
3. comprueba que

$$Rev(y_j) = x_{j+1}.$$

No necesita almacenar las cadenas completas.

Puede recorrer el input varias veces utilizando índices.

Si la entrada tiene longitud n , cualquier posición requiere solamente

$$\lceil \log n \rceil$$

bits.

Por ejemplo, para comparar

$$y_j = a_1 a_2 \cdots a_m$$

con

$$x_{j+1} = b_1 b_2 \cdots b_m,$$

comprueba sucesivamente

$$a_m = b_1,$$

$$a_{m-1} = b_2,$$

etc.

Sólo necesita contadores para indicar las posiciones actuales del input.

Cada contador requiere

$$O(\log n)$$

espacio.

Por tanto,

$$\boxed{L \in NL}.$$

11. Falta demostrar que L es NL -hard

Tomemos una instancia de POINTED-LPATH:

$$A_1, A_2, \dots, A_k$$

con

$$A_1 = \{s\}, \quad A_k = \{t\}.$$

Para cada conjunto de aristas entre dos niveles consecutivos

$$A_i \rightarrow A_{i+1},$$

creamos un bloque B_i .

Si existe una arista

$$u \rightarrow v,$$

introducimos en B_i el par

$$\boxed{(u, Rev(v))}.$$

Por ejemplo,

$$A_1 \rightarrow A_2$$

podría producir

$$B_1 = (s, Rev(a)), (s, Rev(b)).$$

El siguiente nivel:

$$B_2 = (a, Rev(c)), (b, Rev(d)).$$

Y finalmente:

$$B_3 = (c, Rev(t)), (d, Rev(t)).$$

Ahora,

$$s \rightarrow a \rightarrow c \rightarrow t$$

existe si y solo si podemos seleccionar

$$(s, Rev(a)),$$

$$(a, Rev(c)),$$

$$(c, Rev(t)).$$

Las condiciones de L son precisamente

$$Rev(Rev(a)) = a,$$

$$Rev(Rev(c)) = c.$$

Por tanto,

$$\boxed{G \text{ tiene un camino } s \rightsquigarrow t \iff f(G) \in L.}$$

La reducción solamente tiene que recorrer las aristas y escribir

$$(u, Rev(v)).$$

Para producir $Rev(v)$ no necesita guardar v completo: puede recorrer su codificación de derecha a izquierda usando un contador de $O(\log n)$ bits.

Luego

$$POINTED-LPATH \leq_L L.$$

Como POINTED-LPATH es NL -hard,

$$\boxed{L \text{ es } NL\text{-hard.}}$$

La reducción estándar hace precisamente esta codificación de las aristas mediante pares cuyo segundo componente contiene invertido el nombre del vértice siguiente.  studylib.net

12. Juntamos las tres piezas

Hemos demostrado:

$$L \in CFL$$

porque existe un PDA que lo reconoce.

Además,

$$L \in NL$$

porque una máquina no determinista puede seleccionar los pares y verificar sus coincidencias utilizando índices de tamaño logarítmico.

Finalmente,

$$PATH \leq_L LEVELED-PATH \leq_L POINTED-LPATH \leq_L L.$$

Como PATH es NL -completo,

$$L \text{ es } NL\text{-hard.}$$

Por tanto

$$L \in NL \quad \wedge \quad L \text{ es } NL\text{-hard.}$$

De donde

$$L \text{ es } NL\text{-completo.}$$

Y como además

$$L \in CFL,$$

concluimos:

$$\exists L \in CFL \text{ tal que } L \text{ es } NL\text{-completo.}$$

La intuición que conviene recordar

La construcción funciona por una combinación muy bonita de **no determinismo + pila**. El no determinismo permite **escoger una arista en cada nivel**, mientras que la pila permite verificar que dos aristas consecutivas realmente se conectan:

$$(u, v^R), \quad (v, w^R).$$

Es decir, una palabra aceptada por el CFL actúa como una codificación de:

“existe una sucesión compatible de aristas”

que no es otra cosa que

“existe un camino dirigido”.

Ésa es precisamente la conexión con NL : **PATH es el problema prototípico de la elección no determinista de un camino usando sólo espacio logarítmico.**